

MINI ASSESSMENT - LAB REPORT

Name: **Harihara Sudhan**

Department: CSE (AI & ML)

Register No: **99230041113**

Date: 11-09-2026

1. AIM

To design and develop an interactive, rule-based conversational chatbot in Python using conditional `if-elif-else` branch statements and an interactive execution loop, that collects the user's name and responds appropriately to greetings and topic-based queries.

2. PROCEDURE

1. Prompt the user for their name at the start of the program using `input()`, before the main conversation loop begins.
2. Initialize a continuous execution loop (`while True`) to sustain an ongoing conversation.
3. Prompt and read user input string via `input()` inside the loop.
4. Standardize input format by trimming outer white space with `.strip()` and converting to lowercase with `.lower()`.
5. Evaluate termination keywords ('bye', 'exit') and use `break` to gracefully exit the loop.
6. Use sequential `if-elif` branches with the `in` operator to flexibly detect greetings (e.g. 'hello', 'hi', 'hey' appearing anywhere in the input) and at least three topic queries (status, identity, time, help).
7. Personalize at least one response using the user's stored name.
8. Provide a fallback response in the final `else` block for unrecognized input.

3. FLOWCHART

```
[Start]
|
v
[Ask user's name] --> [Store name]
|
v
[Loop: while True]
|
v
[Read user input]
|
v
[Preprocess: strip() & lower()]
|
v
[Input == 'bye' / 'exit'?] -- Yes --> [Print goodbye with name] --> [break] --> [End]
```

```

    | No
    v
['hello'/'hi'/'hey' in input?] -- Yes --> [Print greeting with name]
    | No
    v
['how are you' in input?] -- Yes --> [Print status]
    | No
    v
['your name' in input?] -- Yes --> [Print identity]
    | No
    v
['time' in input?] -- Yes --> [Print time notice]
    | No
    v
['help' in input?] -- Yes --> [Print help menu]
    | No
    v
[Fallback: else] --> [Print "didn't understand"]
    |
    v
[Loop back to Read user input]

```

4. PYTHON CODE

```

def rule_based_bot():
    name = input("Bot: Hi! What's your name?\nYou: ").strip().title()
    print(f"Bot: Nice to meet you, {name}! Type 'bye' or 'exit' anytime to quit.\n")

    while True:
        user_input = input("You: ").strip().lower()

        if user_input in ["bye", "exit"]:
            print(f"Bot: Goodbye, {name}! Have a great day!")
            break

        elif "hello" in user_input or "hi" in user_input or "hey" in user_input:
            print(f"Bot: Hello {name}! How can I help you today?")

        elif "how are you" in user_input:
            print("Bot: I'm doing great, thanks for asking! How about you?")

        elif "your name" in user_input:
            print("Bot: I'm a simple rule-based chatbot built in Python.")

        elif "time" in user_input:
            print("Bot: Sorry, I can't check the time yet, but I'm learning!")

        elif "help" in user_input:
            print(f"Bot: Sure {name}, you can greet me, ask how I am, ask my name, ask about the time")

        else:
            print("Bot: Sorry, I didn't understand that. Type 'help' to see what I can do.")

if __name__ == "__main__":
    rule_based_bot()

```

5. RESULT

```
Bot: Hi! What's your name?
You: hari
Bot: Nice to meet you, Hari! Type 'bye' or 'exit' anytime to quit.

You: hey hello
Bot: Hello Hari! How can I help you today?
You: how are you
Bot: I'm doing great, thanks for asking! How about you?
You: what is your name
Bot: I'm a simple rule-based chatbot built in Python.
You: what's the time
Bot: Sorry, I can't check the time yet, but I'm learning!
You: help
Bot: Sure Hari, you can greet me, ask how I am, ask my name, ask about the time, or type 'bye' to exit.
You: can you dance
Bot: Sorry, I didn't understand that. Type 'help' to see what I can do.
You: bye
Bot: Goodbye, Hari! Have a great day!
```

6. CONCLUSION

The rule-based conversational chatbot was successfully implemented using conditional branching structures (`if-elif-else`) inside an active `while` loop. The bot collects the user's name, uses the `in` operator to flexibly recognize greetings and topic keywords within longer sentences, personalizes responses using the stored name, and gracefully handles both conversation termination and unrecognized queries through a fallback response.